
Introduction of LLM Steering

Prof. Shou-De Lin (sdlin@csie.ntu.edu.tw)

TA

蔡清育 r14922006@ntu.edu.tw

陳曉熙 r14922052@ntu.edu.tw

Steering Methods

- This lecture follows AISteer360's organization:
 - **Input Control:** Prompt & Input
 - **Structural Control:** Model parameters & Architecture
 - **State Control:** Internal activations & Attentions
 - **Output Control:** Generation & Decoding

Input Control

Input Control for LLMs

Control behavior by changing what the model receives

Prompt

Write better instructions

Examples

Show desired behavior

Retrieval

Choose useful context

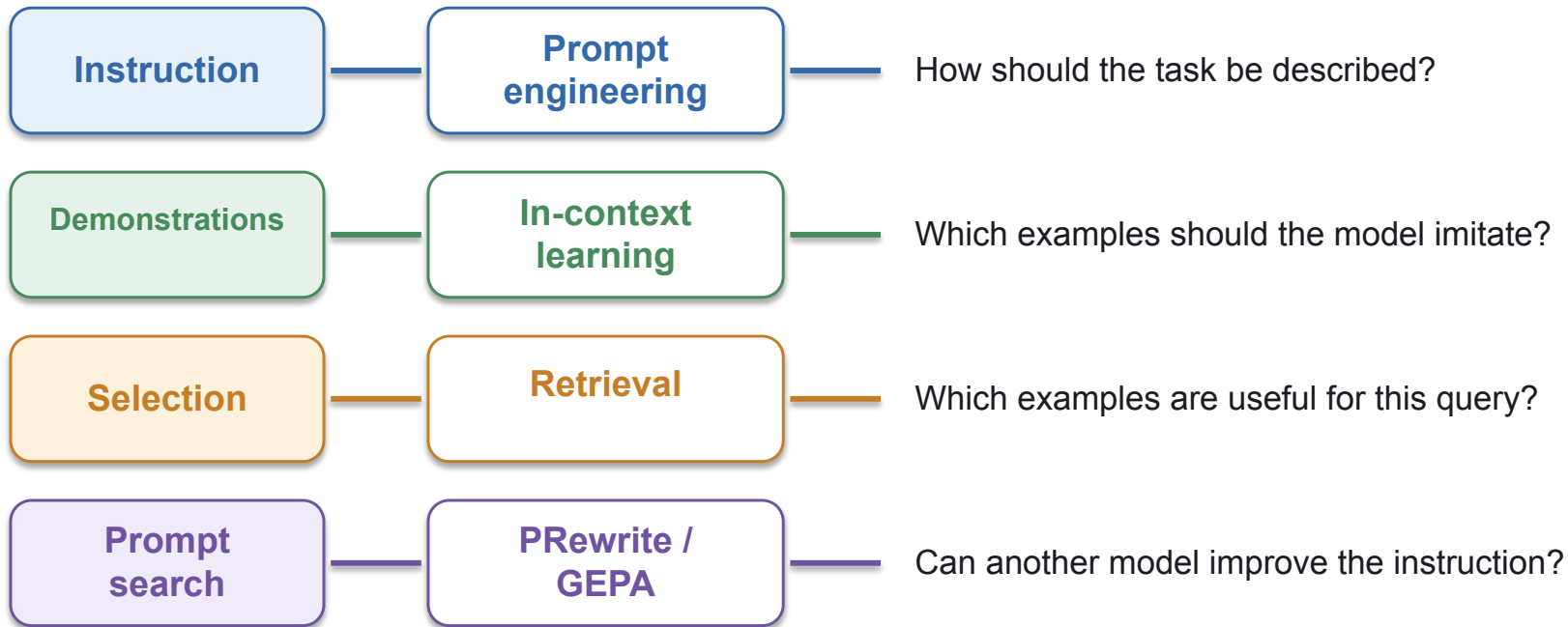
Optimization

Automatically improve prompts

No weight update. No access to hidden states. Works with black-box APIs.

A map of input control

All methods change the input, but they optimize different parts of the context.



Input control is usually the cheapest control layer — so it is often the first place to try.

Prompt engineering: make the task specification explicit

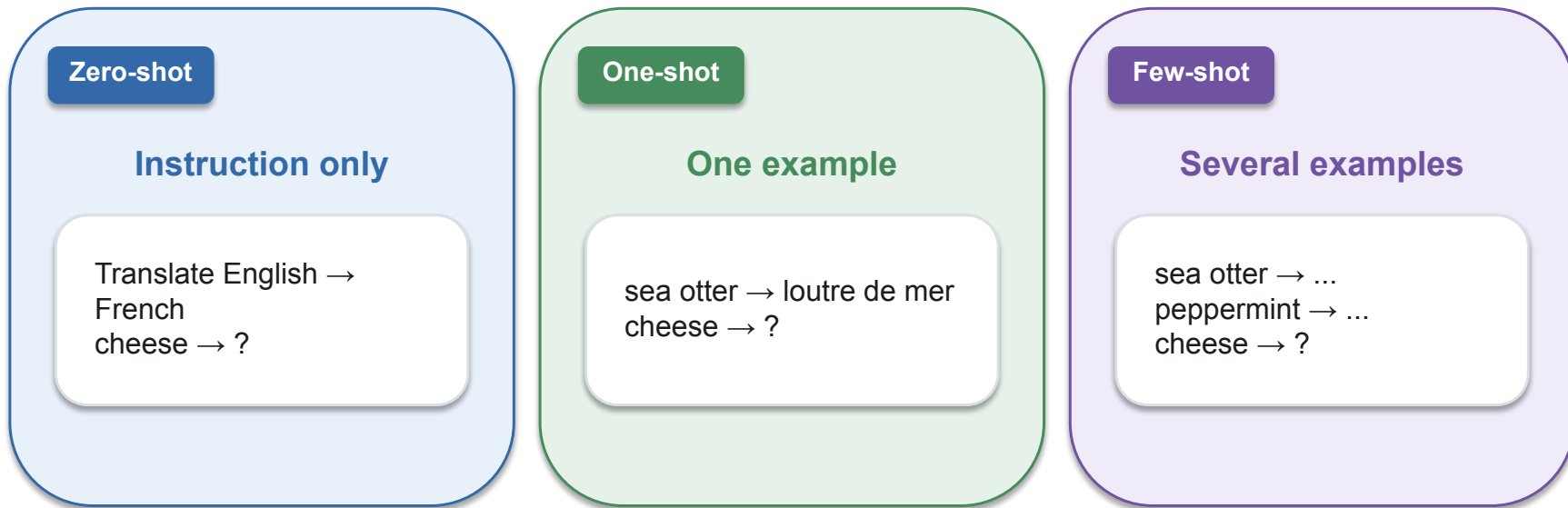
A useful prompt reduces ambiguity about the **goal**, **evidence**, **constraints**, and **output**.

1. Goal	What should the model do?	Classify the review sentiment.
2. Evidence	What information may it use?	Use only the review text below.
3. Constraints	What must / must not happen?	Return one of: positive, neutral, negative.
4. Output	What should the output look like?	Output JSON: {"label": ...}

prompting is interface design — specify the contract between user and model.

In-context learning: show, don't train

Demonstrations teach the behavior inside the context window; the model weights stay frozen.



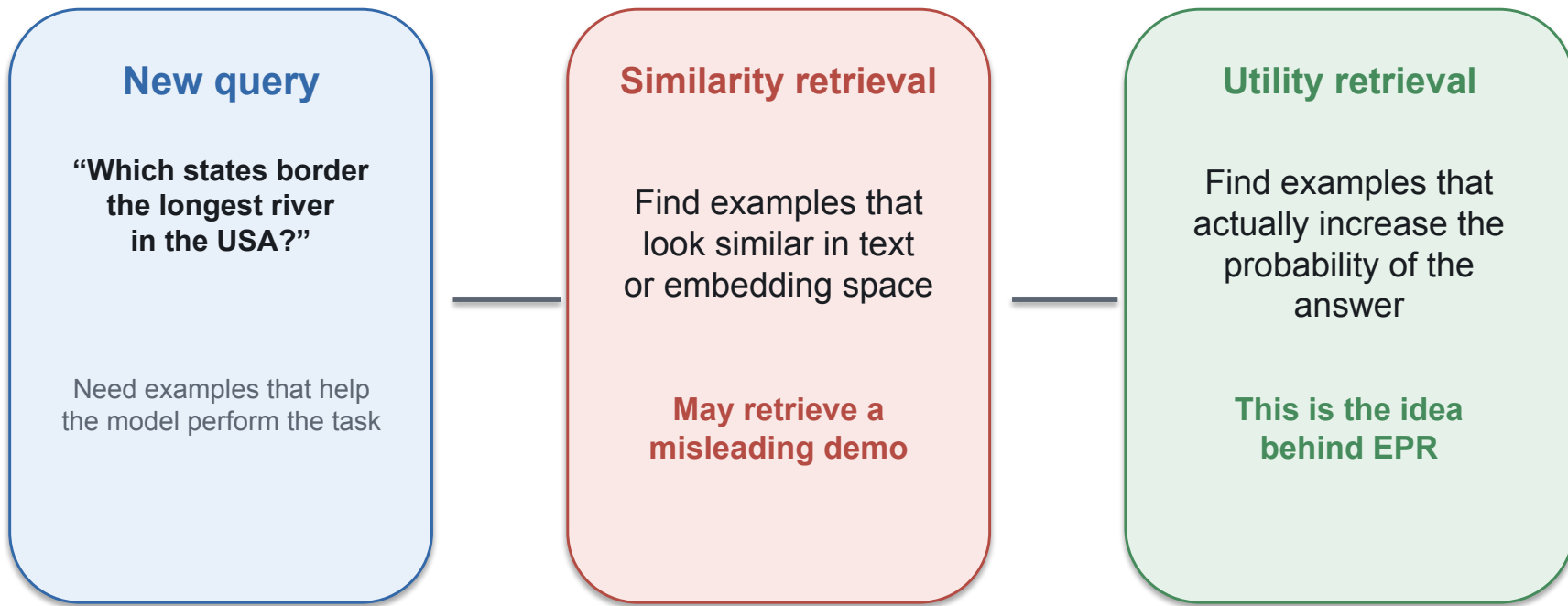
Benefit: easy and black-box.

Risk: behavior can change dramatically with demonstration choice and order.

Brown et al., "Language Models are Few-Shot Learners" (NeurIPS 2020).

The hard part of ICL is choosing the examples

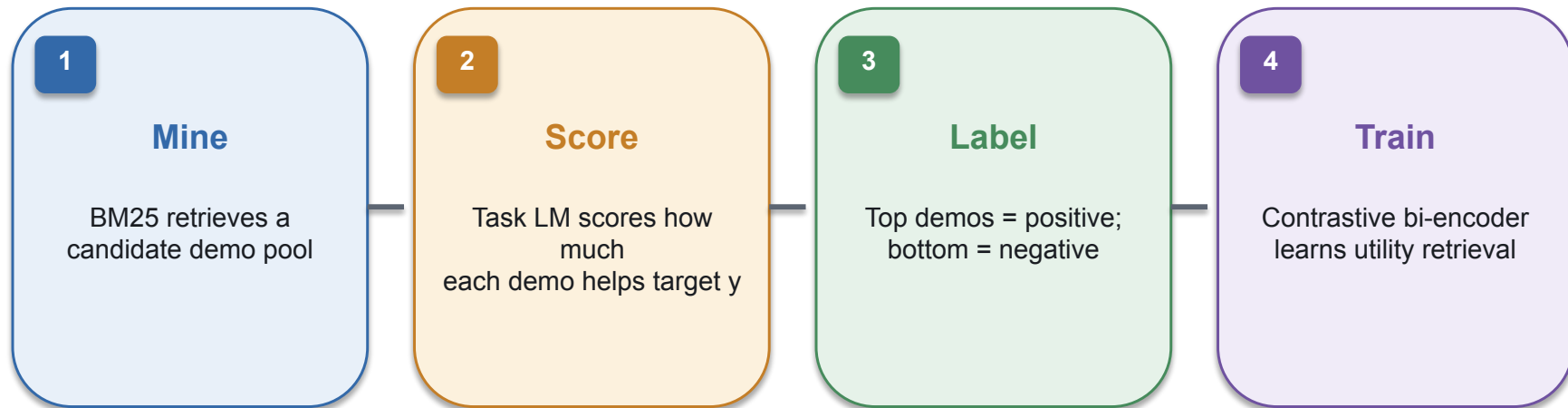
A semantically similar example is not necessarily the example that best helps the LLM answer.



Retrieval objective should match downstream usefulness, not just semantic closeness.

EPR (Efficient Prompt Retrieval): train a retriever for prompt usefulness

Use the task LM itself to tell us which demonstrations are helpful.



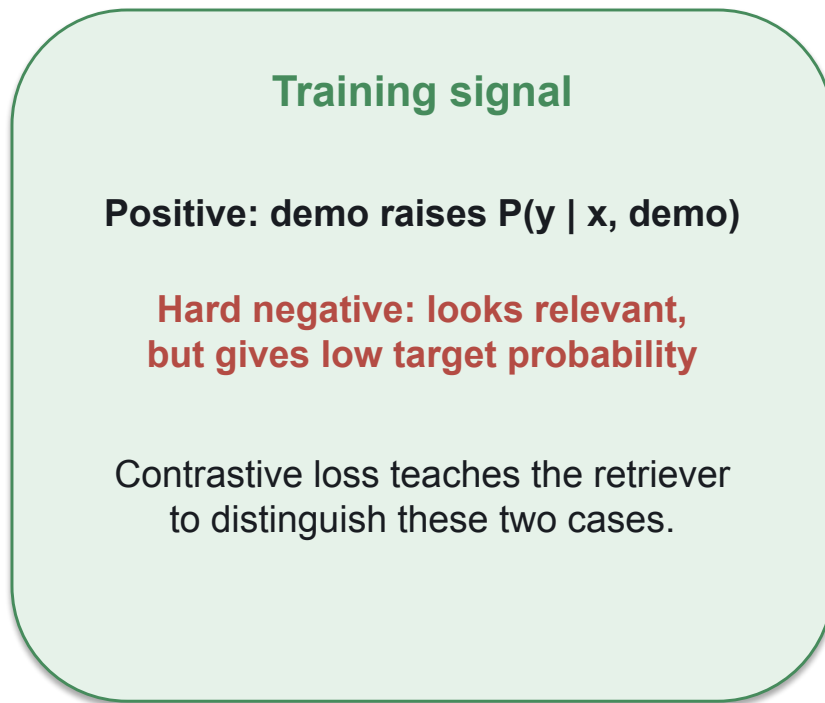
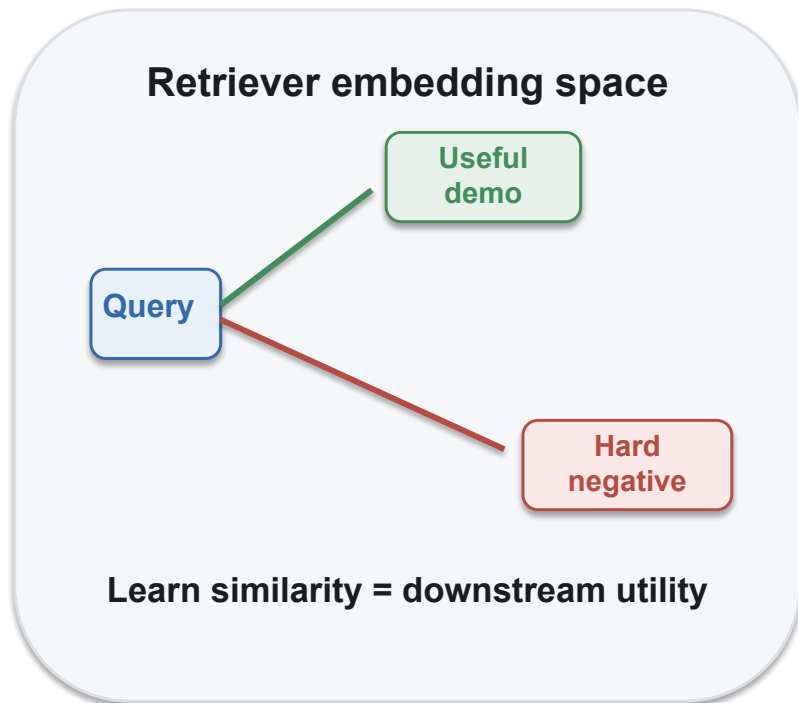
Supervision comes from the inference LM's probability of the correct answer.

So the retriever learns “what helps this LM?” rather than merely “what looks similar?”

Risk: retriever overfits to the teacher LM: “Useful” demonstrations are defined by one task LM.

Why contrastive learning in EPR?

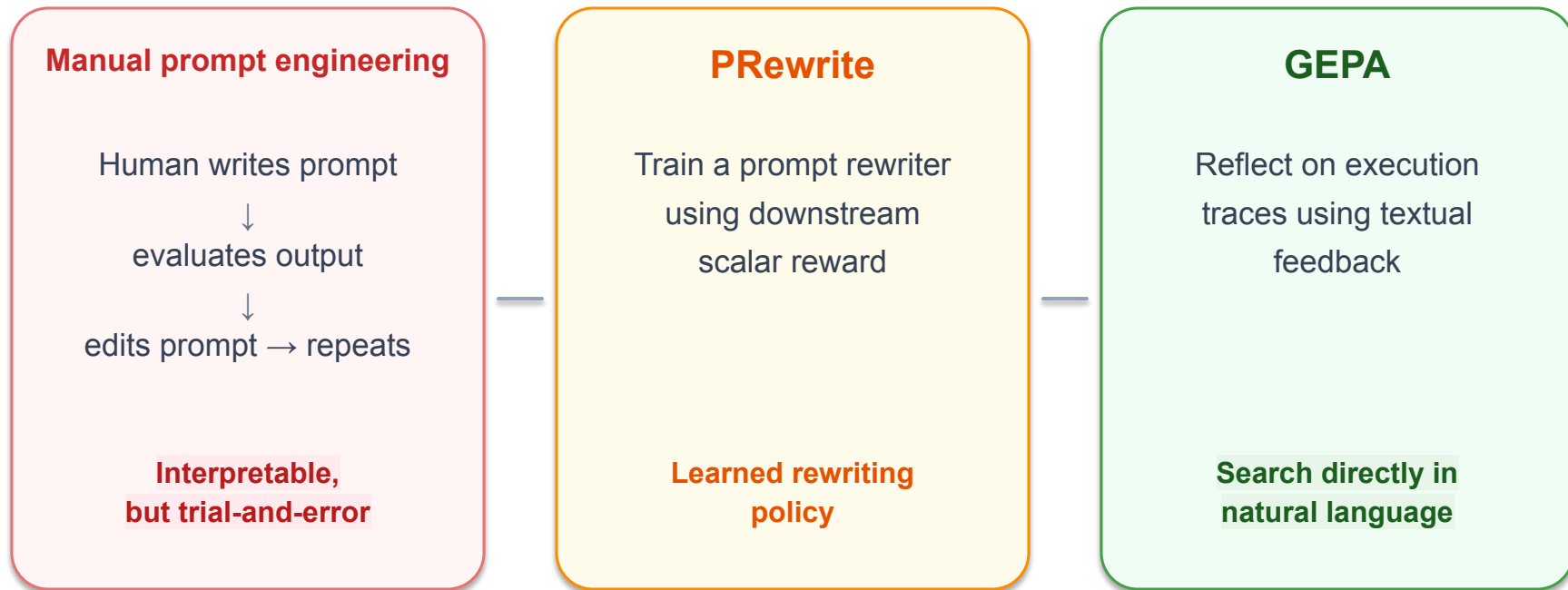
Pull useful query–demo pairs together; push misleading but plausible demos away.



For this lecture, the important idea is functional retrieval — not the exact contrastive-loss derivation.

From manual prompting to automatic prompt optimization

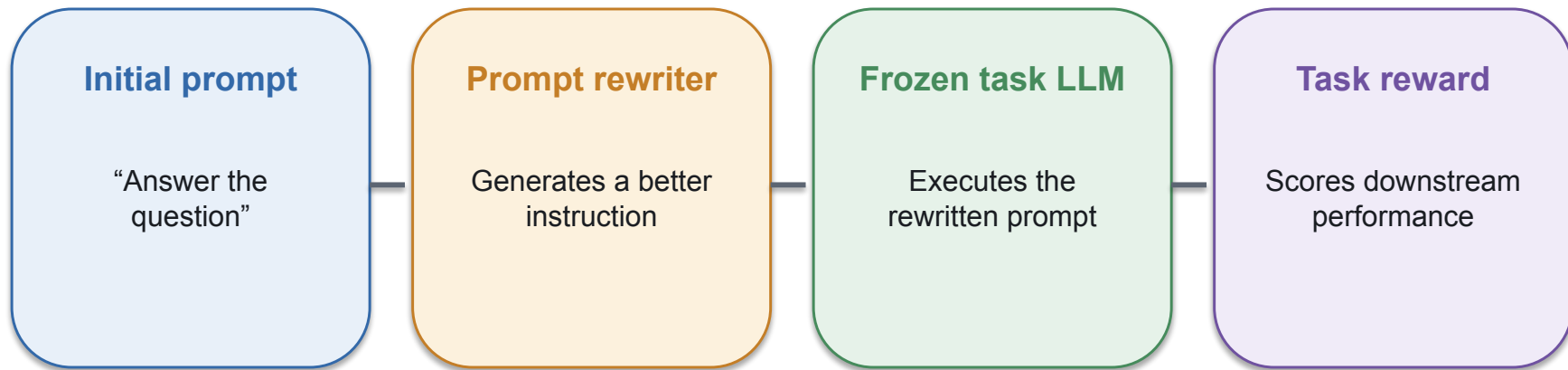
Instead of a human trying many instructions, use another model to search the prompt space.



Different optimization signal — **human judgment**, **scalar reward**, or **textual reflection**.

PRewrite: learn a model that rewrites prompts

The task LLM stays frozen; reinforcement learning trains only the prompt rewriter.



reward trains the rewriter ↻

Important: the task model remains a black box; only its outputs / task score are needed.

Kong et al., "PRewrite: Prompt Rewriting with Reinforcement Learning" (ACL 2024).

PRewrite at inference: one rewrite or search over rewrites?

The paper proposes a deterministic mode and a sampling-based mode.

PRewrite-I

Temperature = 0

Generate one optimized prompt
greedily

Fast at inference
No validation-time search

PRewrite-S

Temperature = 1

Sample multiple rewritten prompts
and select by validation performance

More search
More downstream evaluations

Trade-off: prompt quality vs. evaluation cost.

GEPA: replace scalar-only feedback with reflection

Reflective Prompt Evolution uses language feedback to explain failure and rewrite the prompt.

RL signal

Trajectory $\rightarrow$ reward = 0.2

“It failed.”

The number tells us how bad,
but not what to fix.

GEPA signal

Trajectory $\rightarrow$ textual reflection

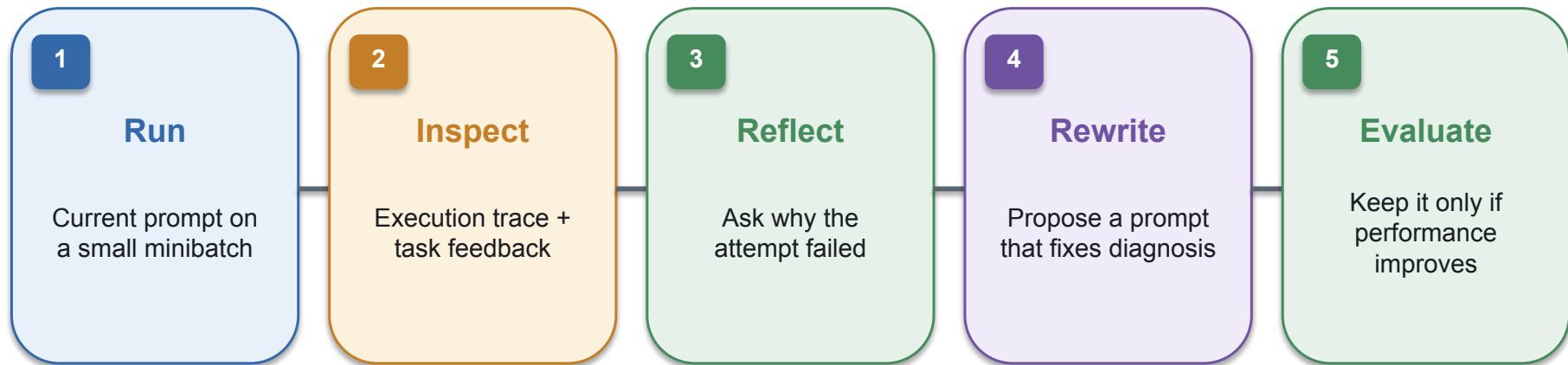
“The prompt misses the constraint and
gives no example of the required format.”

Reflection LLM rewrites the prompt

GEPA is closer to evolutionary optimization than reinforcement learning.

GEPA mutation: learn from the trace, then rewrite

The execution trace becomes evidence for diagnosing what the prompt should change.



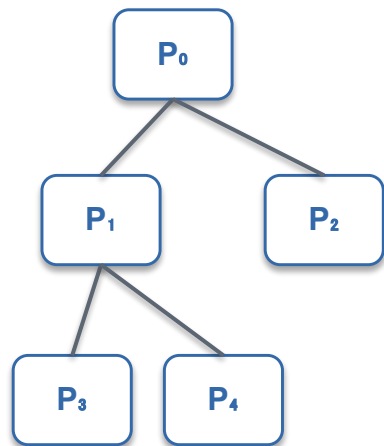
This is prompt optimization, not weight optimization.

GEPA turns “why did this fail?” into an explicit search operator.

Don't bet on one prompt

Maintain multiple branches because different prompts may discover complementary improvements.

Candidate population



Mutation

Pick one branch

Run → inspect trace
→ reflect → rewrite

Merge

Pick complementary branches

Combine useful prompt components into a new candidate

Mutation explores locally; merge recombines strengths found on different branches.

Population search protects against committing too early to one “best-so-far” prompt.

Why “Pareto”? Keep specialists, not only the best average

A prompt survives if it is strongest on at least one task / example.

Toy score matrix

	P ₁	P ₂	P ₃
Task A	0.95	0.70	0.55
Task B	0.60	0.96	0.65
Task C	0.58	0.72	0.94

Keep only the best average?

may discard useful specialists.

Pareto pool

Keep P₁, P₂, P₃ because each is best somewhere.

Diversity gives later mutation and merge better material to work with.

Input control: mini-conclusion

Start by deciding what part of the model's input you want to improve.

Prompt engineering

Write the contract

Clarify task, context, constraints, format

In-context learning

Show examples

Demonstrate the behavior inside the context

EPR

Retrieve useful examples

Optimize demonstrations for downstream utility

PRewrite

Learn to rewrite prompts

Use downstream reward to train a rewriter

GEPA

Reflect and evolve prompts

Use textual feedback + population search

prompt first → examples second → retrieval / automatic optimization when manual design stops scaling.

Structural Control

Structural Control for LLMs

Change the model itself — not just the prompt or context

Fine-tune

Learn a new behavior
from training data

Align

Optimize toward
preferences / rewards

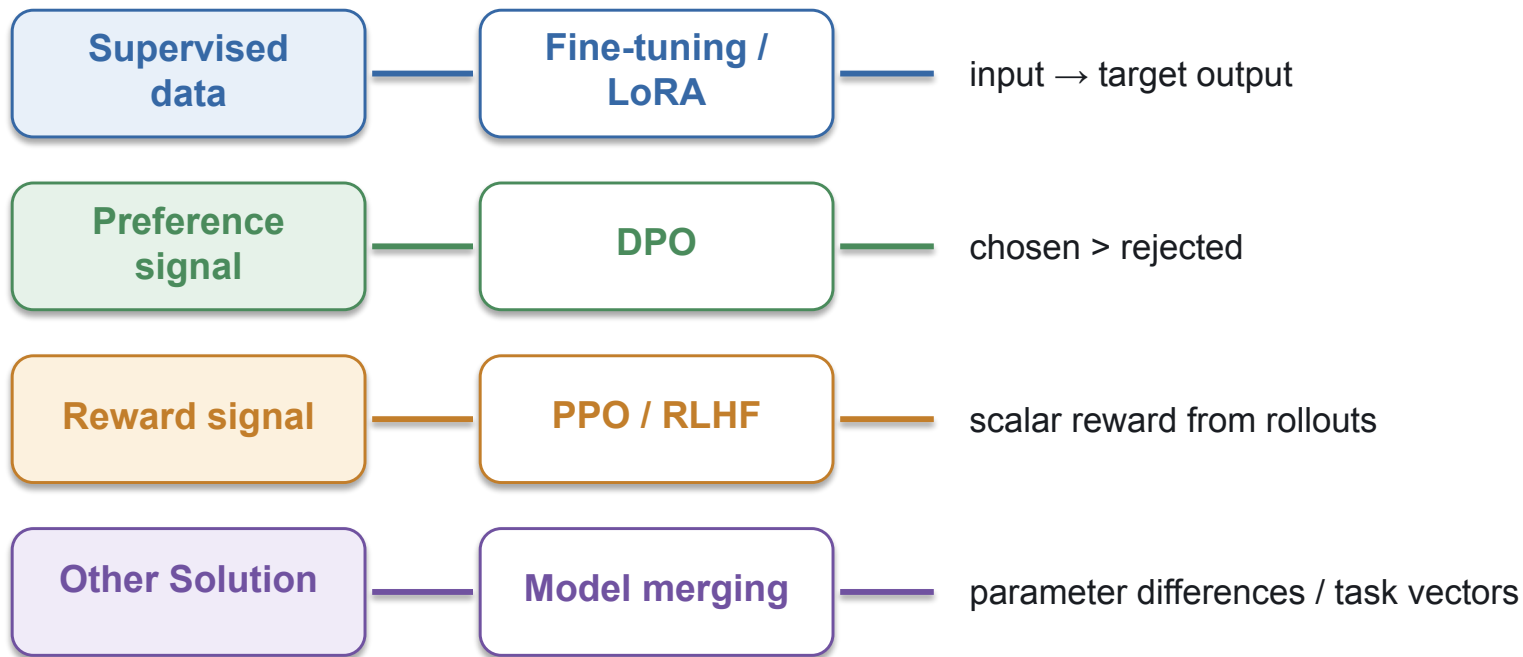
Merge

Combine capabilities
across models

Use structural control when you want a persistent change in behavior, knowledge, or capability.

A map of structural control

The key question is: what signal tells us how the weights should change?



Different supervision signal, cost, stability, and degree of exploration.

Fine-tuning: full update vs. LoRA

Both learn from supervised examples; LoRA changes far fewer trainable parameters.

Full FT

Update all pretrained weights

$$W \rightarrow W + \Delta W$$

- ✓ Maximum flexibility
- ✗ High GPU / memory cost
- ✗ Higher forgetting risk

LoRA

Freeze W ; learn a low-rank update

$$W + BA$$

rank $r \ll d$

- ✓ Far fewer trainable parameters
- ✓ Cheap to store / swap adapters
- △ Capacity constrained by rank

Full FT edits the whole model; LoRA learns a compact patch.

Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models" (2021).

Preference alignment: PPO vs. DPO

Both try to make preferred answers more likely, but they get there differently.

PPO

Learn from rewards online



- + Can explore new response trajectories
- Reward model + reference + value model
- More moving parts; reward hacking risk

DPO

Learn directly from preference pairs

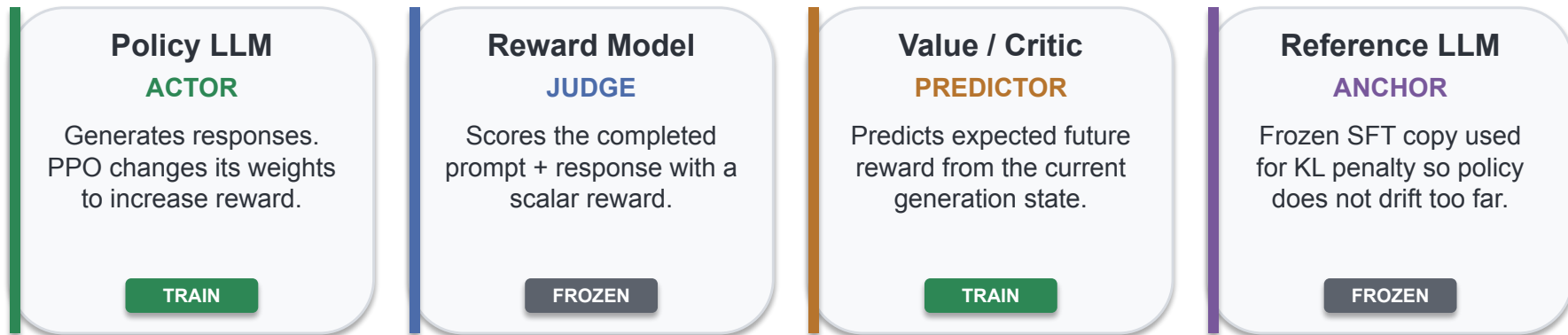


- + Simpler and memory-efficient
- Quality depends heavily on preference data

PPO = online reward optimization. DPO = offline preference optimization.

PPO: Four Models — Which Ones Are We Training?

The policy is the LLM we want to improve; the critic is an auxiliary predictor.



Judge: evaluates the completed outcome

Critic: predicts the expected outcome

Reward Model gives the target signal; Critic learns $V(s_t)$ so PPO can estimate advantage.

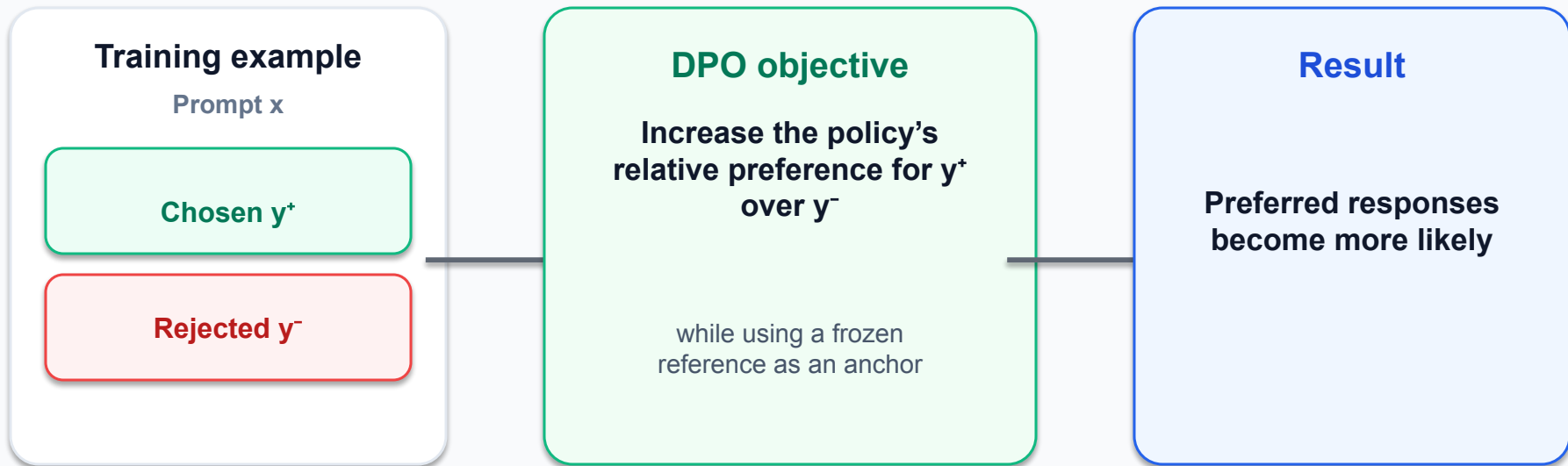
During PPO: TRAIN Policy + Critic

FROZEN Reward Model + Reference LLM

Why PPO is hard: four-model infrastructure, on-policy sampling cost, unstable optimization, and reward-model blind spots.

DPO: turn preference learning into a classification-like objective

No separate reward model and no online RL loop.

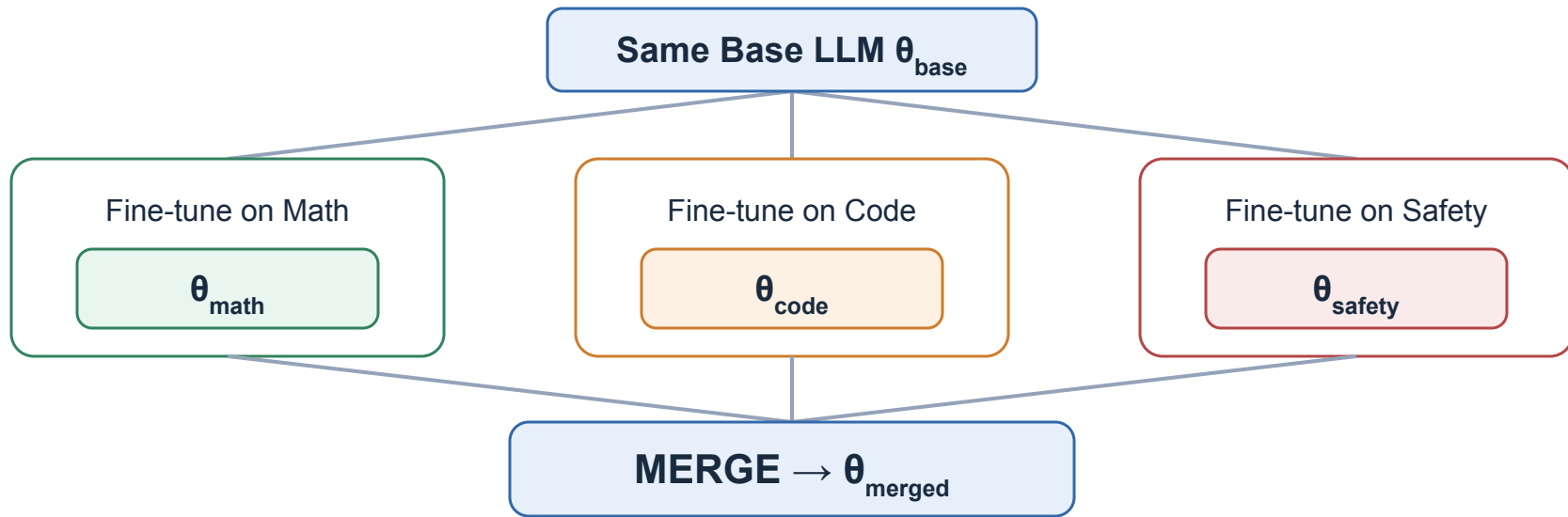


Best when you already have reliable preference pairs; sensitive to noisy or biased comparisons.

Model Merging: combine capabilities without retraining

1/2

Start with several specialists derived from the same base model.



Goal: reuse capabilities already learned

Why it is attractive? no original training data required

Model merging changes the weights by combining learned weight updates — not by training again on all tasks.

Simple Model merging

Treat each fine-tune as a direction in parameter space.

Task vector

$$\tau_A = \theta_A - \theta_{\text{base}}$$

What changed when the base model learned Task A?

Task arithmetic

$$\theta_{\text{new}} = \theta_{\text{base}} + \alpha \tau_A + \beta \tau_B$$

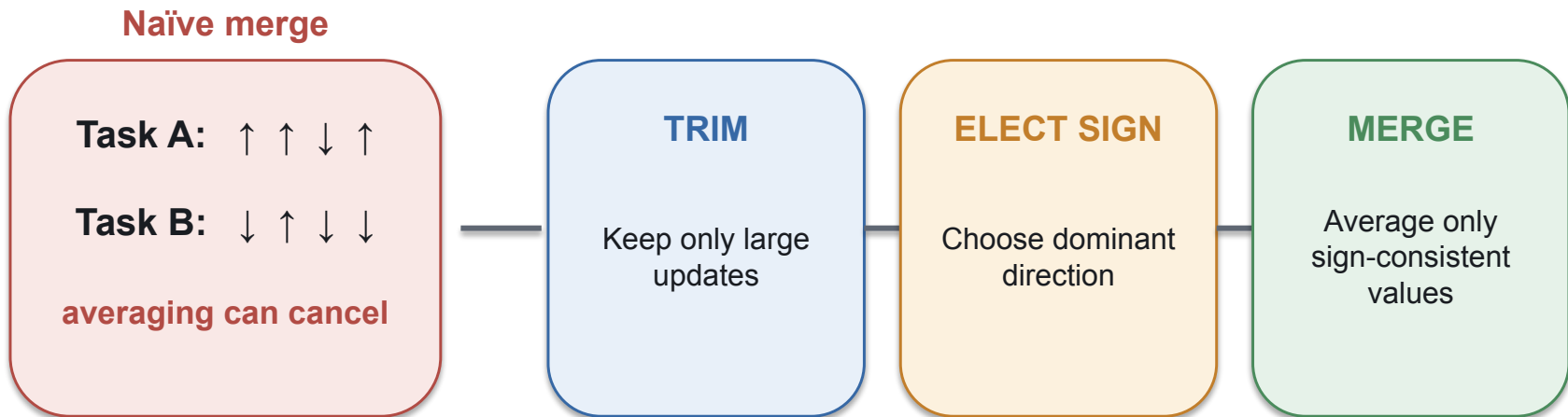
Add directions to combine skills;
subtract a direction to suppress a
behavior.

Attractive because it is training-free — but task vectors can interfere.

Ilharco et al., "Editing Models with Task Arithmetic" (2022).

TIES-Merging: resolve interference before averaging

Problem: different task vectors may update the same parameter in opposite directions.



TIES = remove redundant small changes → settle sign conflicts → merge what agrees.

Trade-off: better multi-model merging, but introduces density K and scaling λ to tune.

Yadav et al., "TIES-Merging: Resolving Interference When Merging Models" (NeurIPS 2023).

Which structural-control method should I use?

Choose based on the supervision you have and the kind of change you need.

Method	Signal / data	Training cost	Strength	Main risk
Full FT	Input → target	High	Maximum adaptation	Cost / forgetting
LoRA	Input → target	Low–Med	Cheap modular adaptation	capacity limits
DPO	Chosen vs rejected	Medium	Simple preference alignment	Preference-data noise
PPO	Reward on rollouts	Very high	Online exploration	Complexity / reward hacking
Merge / TIES	Fine-tuned Model weights	Low	Combine models without retraining	Parameter interference

Rule of thumb: start with the cheapest method that matches your supervision; pay PPO's complexity only when online exploration is actually valuable.

State Control

Where state control fits

Structural control changes weights; state control changes activations; input control changes what the model sees.

Input control

WHAT CHANGES?

Prompt / context

WHEN?

Before inference

- Easy to deploy

Structural control

WHAT CHANGES?

Model weights

WHEN?

Before deployment

- Persistent change

State control

WHAT CHANGES?

Hidden activations

WHEN?

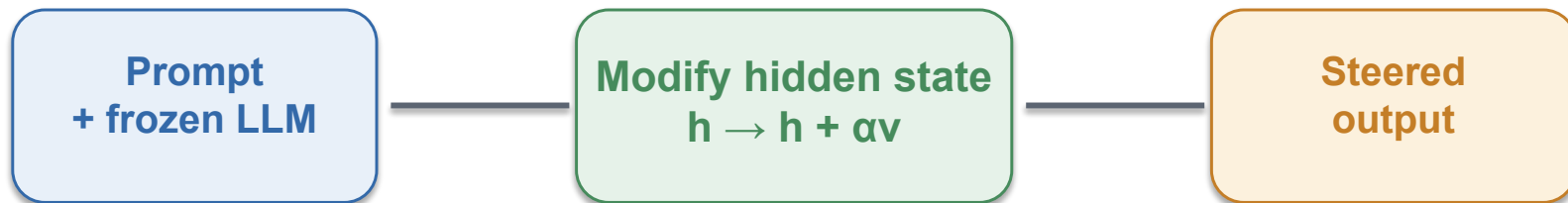
During inference

- No retraining

State control is attractive when you want controllability without producing a new model weights.

State Control for LLMs

Steer the model's internal activations at inference time

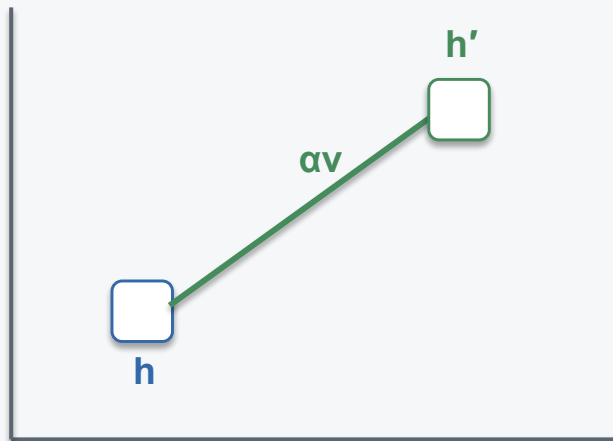


No weight update. No prompt rewrite. The intervention happens inside the forward pass.

The core operation: move the hidden state

A steering vector v represents a direction associated with a desired property.

Conceptual activation space



$$h' = h + \alpha v$$

v = steering direction

α = steering strength

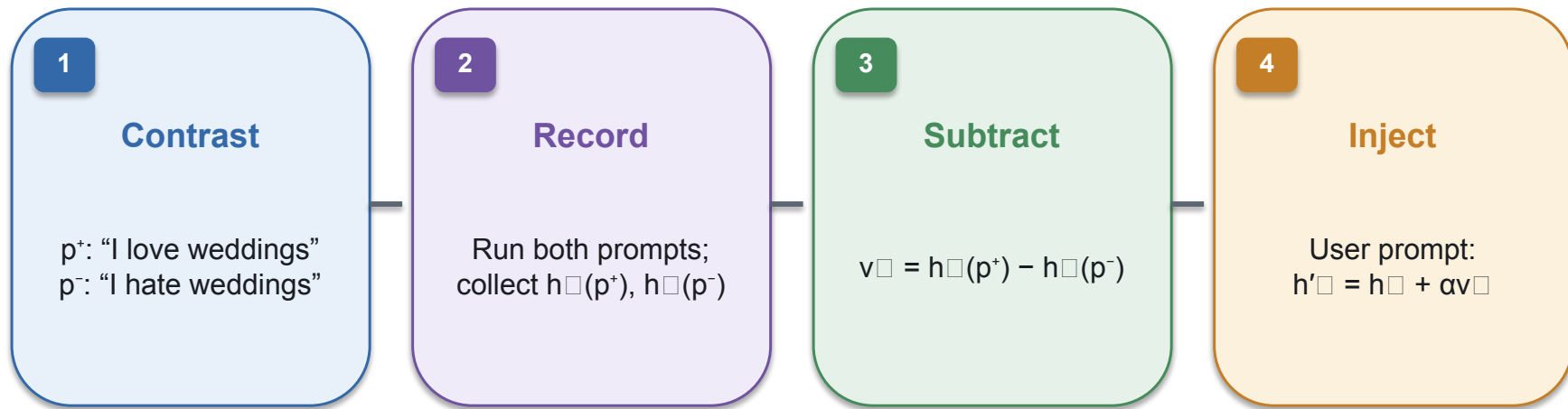
Two practical choices dominate:

- ① where to intervene
- ② how strongly to steer

Most methods differ mainly in how they estimate v and how strong they inject it.

ActAdd: one contrastive pair gives a steering direction

The simplest recipe: subtract activations, then add the difference during inference.



Special case: use a blank negative prompt to obtain a topic direction, e.g. "Wedding" – "".

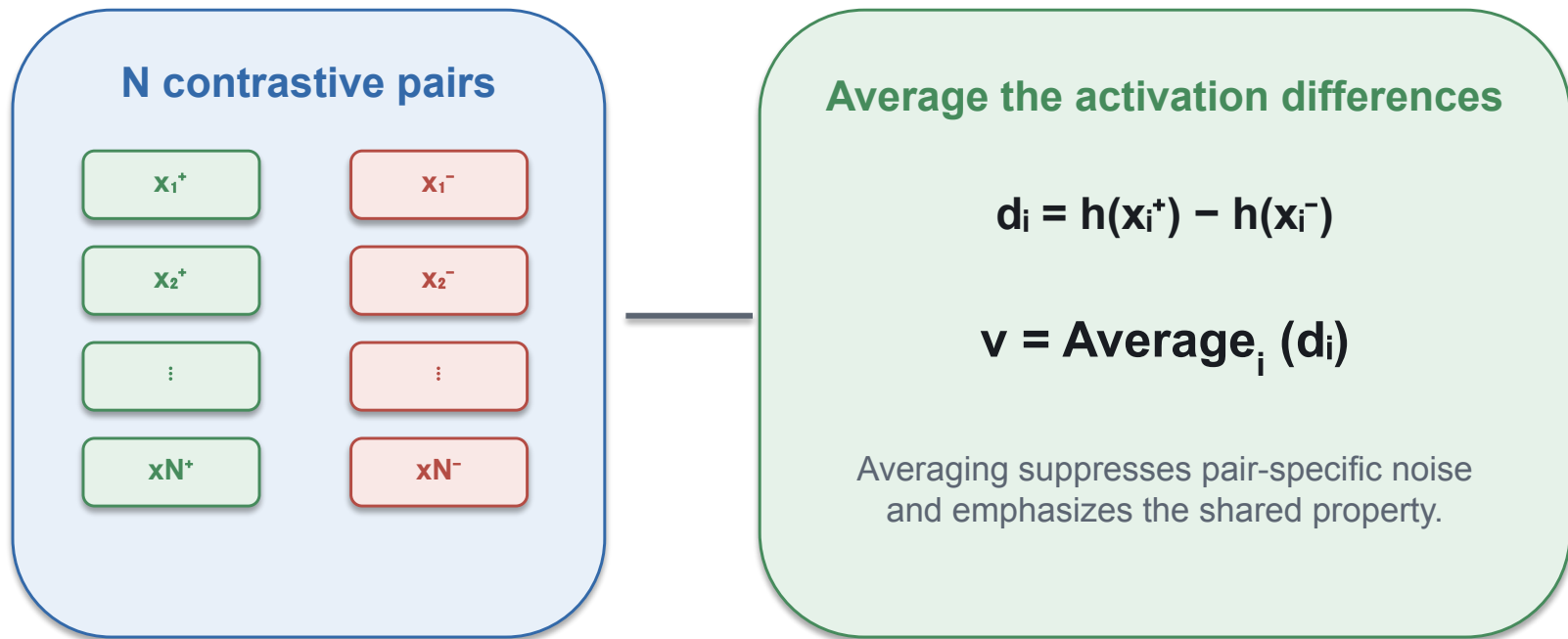
- ✓ No training or dataset
- ✓ Very easy to prototype

- △ Sensitive to prompt pair
- △ Layer and α need tuning

Turner et al., "Activation Addition: Steering Language Models Without Optimization" (2024).

CAA: average many contrasts to get a cleaner direction

ActAdd uses one pair. CAA uses a labeled contrastive dataset.

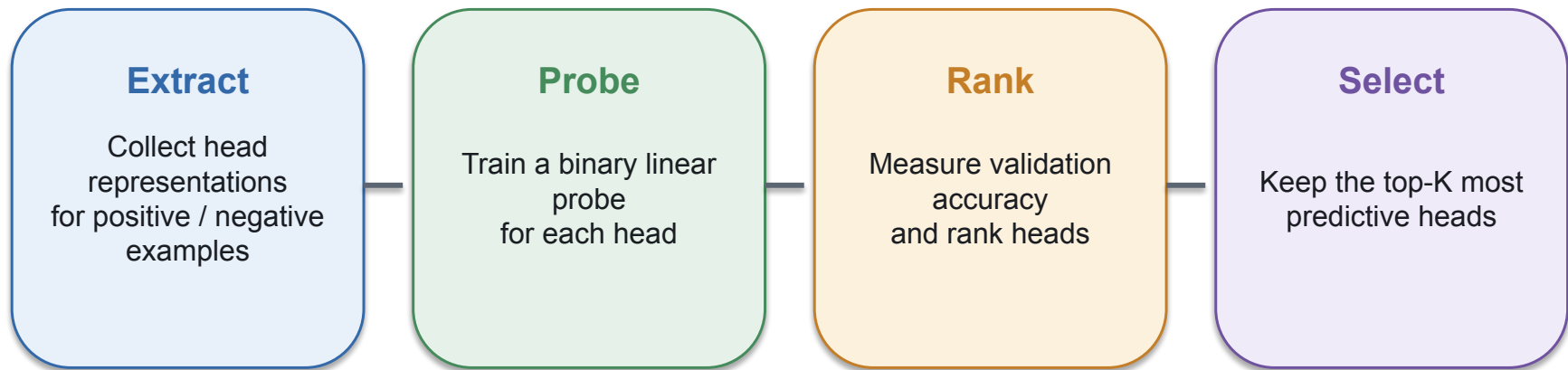


Trade-off: more robust direction, but now you need labeled contrastive data.

Panickssery et al., "Steering Llama 2 via Contrastive Activation Addition" (ACL 2024).

ITI (step 1): locate the heads that encode the property

Train a tiny linear probe for every layer $\times$ attention head.

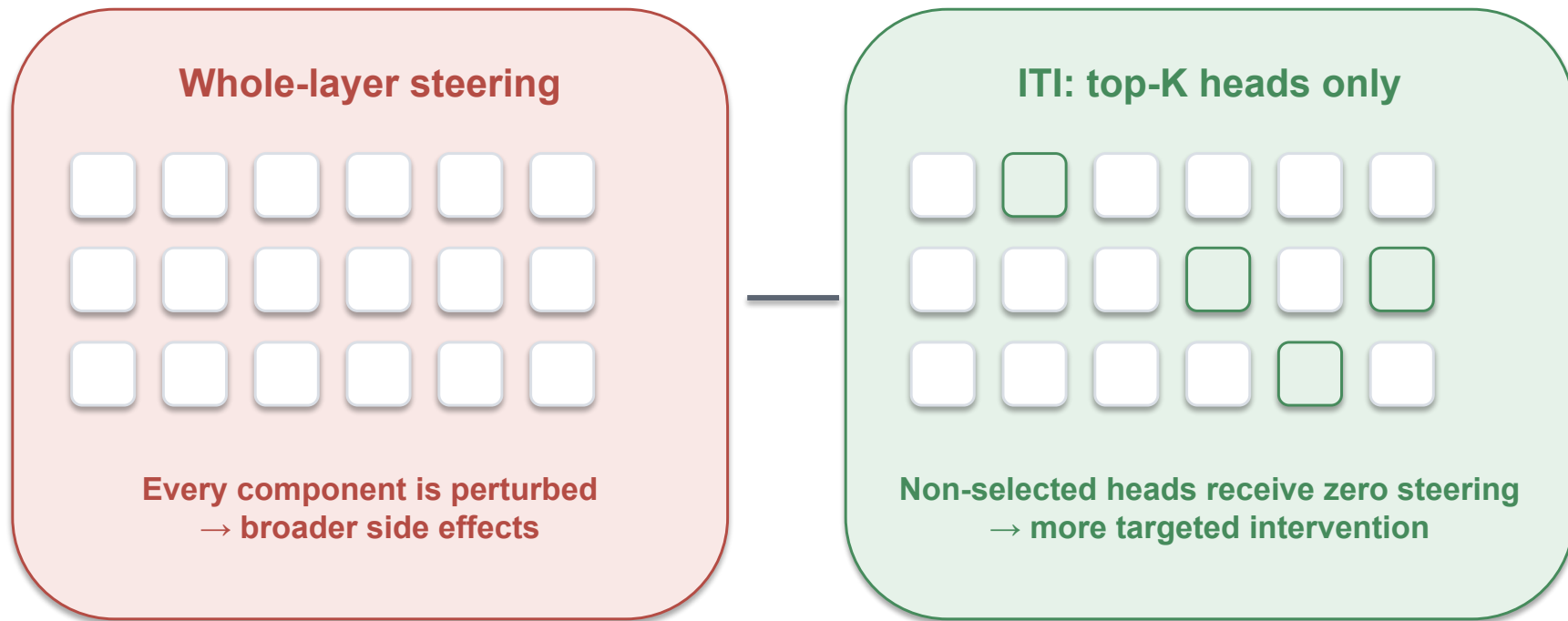


Interpretation

If only a few heads predict the property well, the representation is localized.
If many heads do, the property is distributed.

ITI (step 2): steer only the selected heads

Selective intervention aims to change the target behavior with less collateral damage.



Cost: labeled data + many probes. **Benefit:** interpretable localization and potentially smaller capability degradation.

ActAdd → CAA → ITI: what problem does each step solve?

The methods progressively add robustness and selectivity.

ActAdd

How is v estimated?

One prompt pair

Where do we intervene?

whole chosen layer

Fastest, but noisy

CAA

How is v estimated?

Many prompt pairs

Where do we intervene?

whole chosen layer

Cleaner direction

ITI

How is v estimated?

**Many labeled examples
+ probes**

Where do we intervene?

selected heads only

Cleaner + localized

ITI asks a new question: which internal components actually encode the property?

CAST: steering should depend on the query

A refusal vector is useful for harmful requests — but not if applied to everything.

Input	Always steer?	Desired behavior
Harmful question	✓ Yes	Refuse
Harmless question	✗ No	Answer normally
Math question	✗ No	Solve the problem

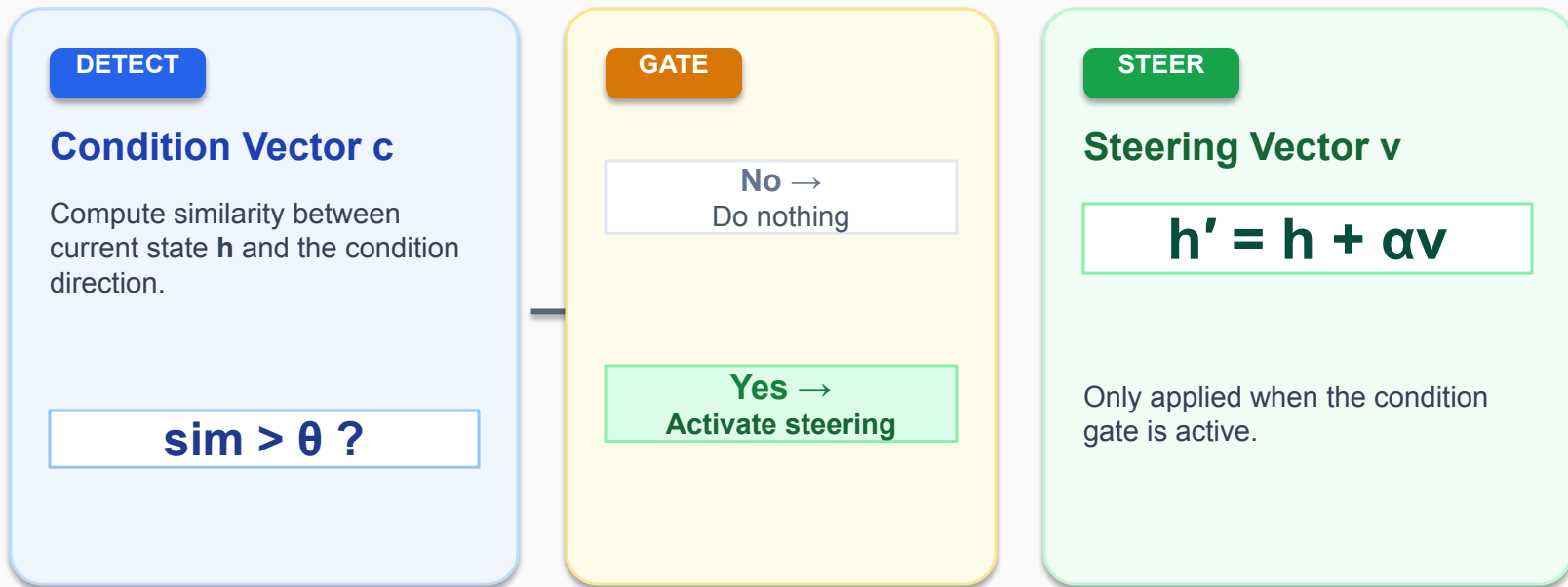
Problem with unconditional steering: the same vector is injected even when the target behavior is irrelevant.

CAST adds a gate: detect whether the input matches the condition, then steer only if needed.

Lee et al., “Programming Refusal with Conditional Activation Steering” (ICLR 2025).

CAST = Condition Vector + Steering Vector + Threshold

Separates “Should I intervene?” from “How should I change the state?”



θ controls false positives vs. false negatives • α controls how strongly the activated behavior is expressed

State-control methods: one progression

From a single global direction to data-driven, localized, conditional steering.

Method	How v is learned	Where / when	Needs labeled data?	Main trade-off
ActAdd	1 contrastive pair	Chosen layer / always	No	Simple, prompt-sensitive
CAA	Average many pairs	Chosen layer / always	Yes	Robust direction
ITI	Mean contrast + probes	Top-K heads / always	Yes	Localized, probe cost
CAST	Steering + condition vectors	Only if condition fires	Yes	Conditional, threshold tuning

The design questions are always the same:

What direction? • Where to inject? • How strong? • Should it fire on this input?

Output Control

Output Control for LLMs

Control how a response is produced — without changing the prompt or model weights

Reweight

Change next-token scores

Select

Generate many, choose one

Budget

Control how long the model reasons

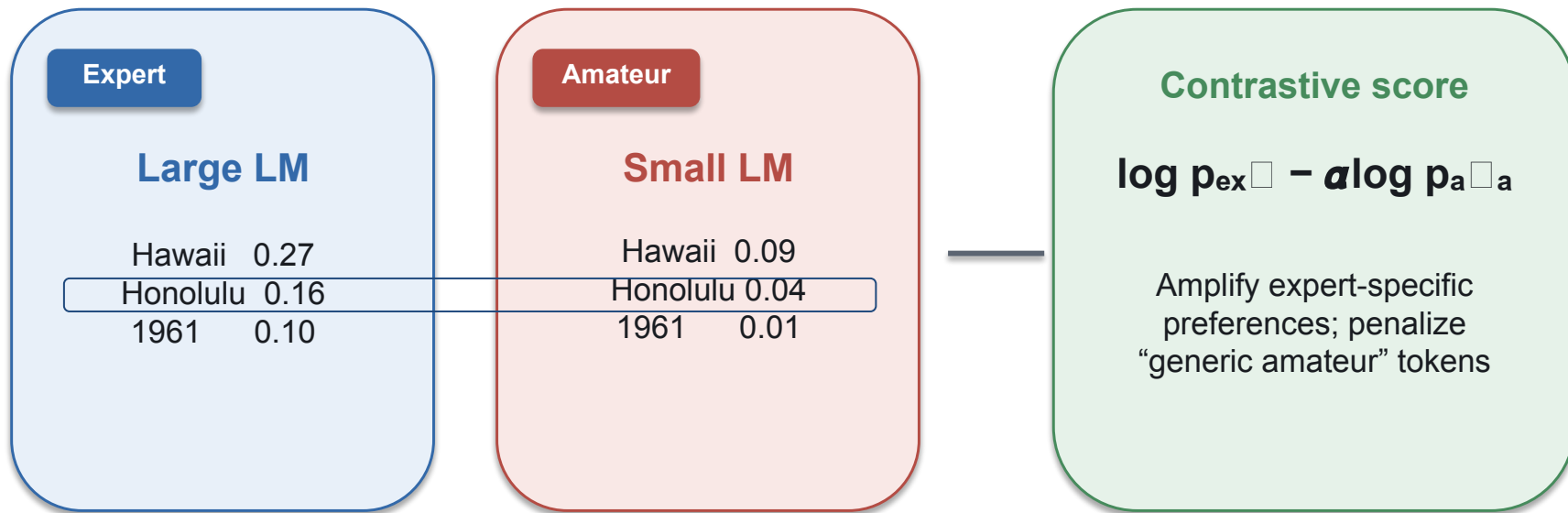
Constrain

Forbid invalid outputs

Output control operates at inference time: change token probabilities, candidate selection, reasoning budget, or validity.

Contrastive Decoding: What does the expert know beyond the amateur?

Prefer tokens the large model likes for reasons the smaller model cannot explain.



No training or parameter update — but every decoding step runs two models.

Concern: need to know in advance which model is an expert/amateur given this query

Li et al., “Contrastive Decoding: Open-ended Text Generation as Optimization” (2023).

DExperts: Logit_{base + expert - anti-expert}

Steer a frozen base LM toward one attribute and away from its opposite.

Base LM

Expert = fine-tune on desired data

Anti-expert = fine-tune on undesired data



$$L_{\text{final}} = L_{\text{base}} + \alpha \cdot (L_{\text{expert}} - L_{\text{anti-expert}})$$

Benefit: preserve the base model

Cost: run three models at once

Liu et al., "DExperts: Decoding-time Controlled Text Generation with Experts and Anti-Experts" (ACL 2021).

Contrastive Decoding vs. DExperts

Both combine logits at decoding time, but the source of the contrast is different.

Contrastive Decoding

expert LM – amateur LM

Goal: improve open-ended generation

2 models

No base-model weight update

DExperts

base + expert – anti-expert

Goal: steer a chosen attribute

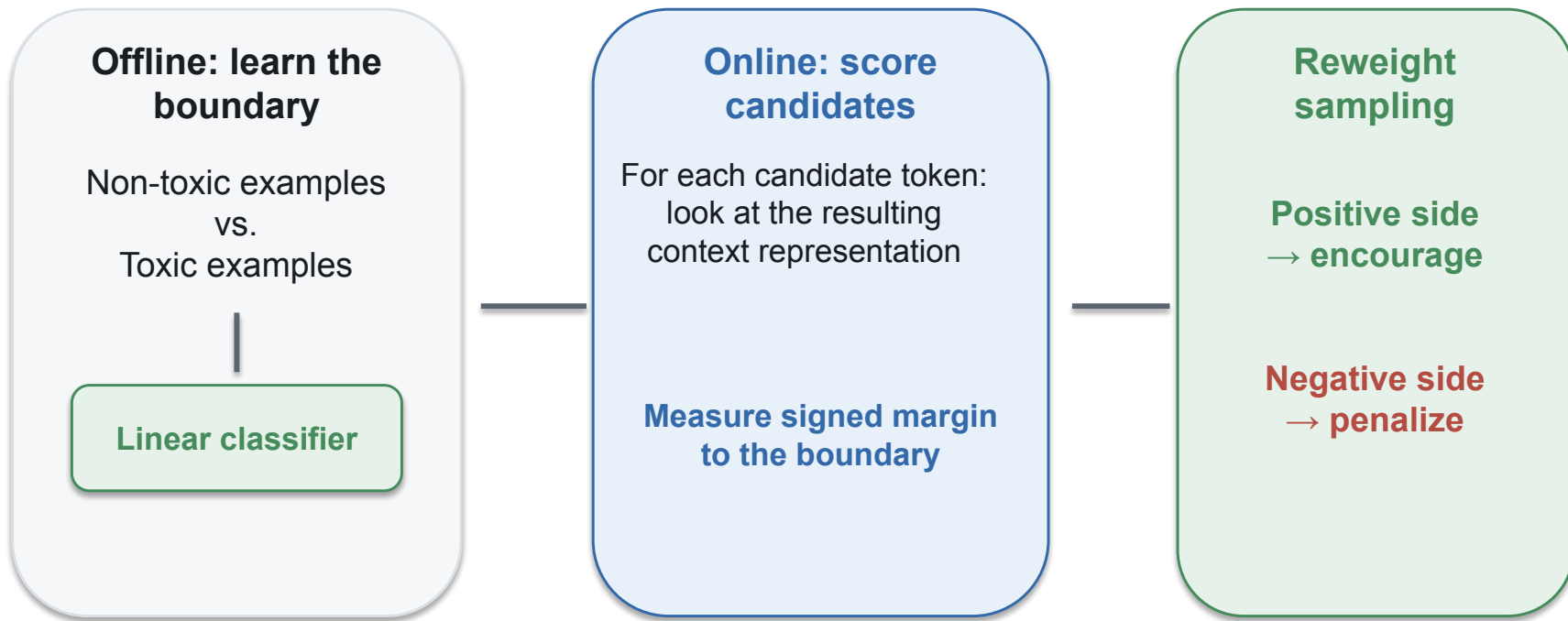
3 models

No base-model weight update

Shared idea: alter the next-token distribution by adding or subtracting another model's preference.

SASA: let the model alter its own generation

Learn a lightweight boundary in the base model's representation space, then use it during sampling.



Ko et al., "Large Language Models Can Be Strong Self-Detoxifiers" (2024).

SASA: one decoding step example

Goal: steer “You are such a ...” away from toxicity using the base LLM’s own hidden representations.

1 Normal decoding

Context
“You are such a”



Base LLM

Next-token
distribution

stupid	.30
nice	.20
great	.15

Take top-K
candidates

2 Candidate look-ahead

Reuse the same context KV cache;
append each candidate for one
token.

+ stupid

→ h_{stupid}

+ nice

→ h_{nice}

+ great

→ h_{great}

These K one-token evaluations
can be batched.
No second LLM is needed.

3 Score hidden states

**Linear attribute
boundary**

$$m(h) = w^T h + b$$

$h_{\text{stupid}} \rightarrow -2.1$

$h_{\text{nice}} \rightarrow +1.3$

$h_{\text{great}} \rightarrow +1.7$

negative = toxic side
positive = non-toxic
side

4 Reweight & sample

Use margins to modify
the original token
probabilities.

stupid .30 ↓ .06

nice .20 ↑ .31

great .15 ↑ .34

**Sample →
“great”**

Append token →
repeat

Key cost: obtaining $h_{\text{candidate}}$ requires extra one-token transformer passes (batched over K candidates); the linear classifier itself is cheap.

SASA's trade-off: lightweight steering vs. fluency

It avoids secondary LMs, but strong probability reweighting can distort generation.

Pros

One base LM + lightweight classifier

Self-contained at inference time

No expert / anti-expert model

Cons

Needs an offline subspace-learning phase

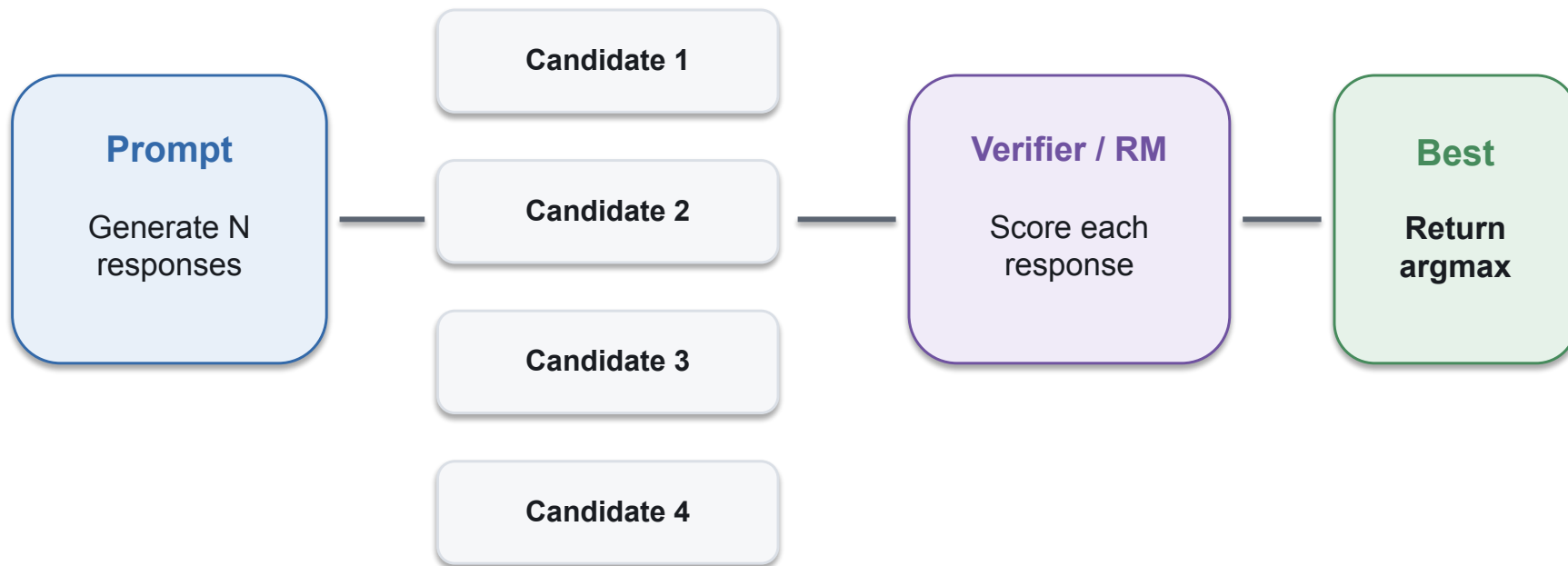
Quality depends on the learned boundary

Aggressive steering can hurt fluency

Compared with DExperts: less inference compute, but more dependence on internal representations.

Best-of-N: spend compute on candidates, then select

Instead of steering every token, generate several complete answers and choose the strongest one.



Very simple test-time scaling: quality often improves with N, while latency / compute grows roughly linearly.

Risk: A weak verifier may select the wrong answer.

Budget Forcing: control how long a reasoning model thinks

Intervene on the stopping behavior rather than on the model weights.

Shorten

Impose a maximum thinking budget

Append end-of-thinking token
or “Final Answer:”

**Benefit: cap latency /
prevent endless reasoning**

Lengthen

Suppress an early stop

Optionally append “Wait”
to continue the reasoning trace

**Can trigger self-correction
on difficult problems**

Risk: A fixed budget may overthink easy problems or underthink hard ones.

Constrained Decoding: only allow tokens that keep the output valid

The LLM still proposes probabilities normally; a constraint engine masks choices that would violate the required structure.

Example goal: valid JSON

Schema: {"name": string, "age": integer}



Result: the model must continue with something compatible with "age": <integer> — invalid tokens are impossible to sample.

$\text{masked_logit}(v) = \text{logit}(v)$ if v is valid in the current parser state; $-\infty$ otherwise

Which output-control method should I use?

Choose based on what you want to control and what inference resources you can afford.

Improve generation quality	Contrastive Decoding	Can run expert + amateur
Steer an specific attribute	DExperts	Can run 3 models
Steer an attribute cheaply	SASA	Have internal representations + offline labels
Spend extra test-time compute	Best-of-N	Have a verifier / reward model
Control reasoning latency	Budget Forcing	Use a reasoning model
Guarantee output syntax	Constrained Decoding	Have a grammar / schema

There is no single best decoder: output control is a compute–control–quality trade-off.